

Lab 4: Exploring WSL—Ubuntu CLI Fundamentals and Command-Line Arguments in C

Theory

Windows Subsystem for Linux (WSL) allows a Linux distribution such as Ubuntu to run within the Windows operating system. It provides a Linux command-line environment for creating directories, managing files, editing programs, compiling source code, and executing applications. The basic WSL commands used in this lab are:

1. `pwd` – displays the current working directory.
2. `cd` – changes the current directory.
3. `mkdir` – creates a new directory.
4. `ls` – lists files and folders in a directory.
5. `nano` – opens a text editor to create or edit files.
6. `gcc` – compiles a C source file into an executable program.
7. `./test` – executes the compiled program named test.
8. `touch` – creates an empty file.
9. `cp` – copies files or directories.
10. `clear` – clears the terminal screen.

Write a C program to demonstrate the use of command-line arguments by adding two numbers passed through the command line.

In C programming, command-line arguments allow users to pass data to a program when it is executed from the terminal. These arguments are received through the `main()` function using the following syntax:

```
int main(int argc, char *argv[])
```

- `argc` (argument count) stores the total number of command-line arguments passed to the program, including the program name.

- `argv` (argument vector) is an array of strings that stores the command-line arguments. In this program, `argv[1]` and `argv[2]` contain the two numbers entered by the user.

Additionally,

- `atoi()` is a library function that converts a string into an integer. It converts the values in `argv[1]` and `argv[2]` so that they can be added mathematically.

Write a C program to demonstrate the use of command-line arguments in Ubuntu WSL by creating, editing, compiling, and executing a C program using the Linux command-line interface.

Procedure:

Step 1: Open Ubuntu in Windows Subsystem for Linux (WSL).

Step 2: Display the current working directory using: `pwd`

Step 3: Navigate to the Home directory (if required): `cd ~`

Step 4: Create a new directory: `mkdir test`

Step 5: Move into the newly created directory: `cd test`

Step 6: Create a C source file: `nano helloworld.c`

Step 7: Write the C program that accepts and displays command-line arguments, then save the file.

Step 8: Compile the program using the GCC compiler: `gcc helloworld.c -o helloworld`

Step 9: Execute the compiled program with command-line arguments: `./helloworld`

Step 10: Observe the output displaying the total number of arguments and their values.

Step 11: End the program.

Source Code:

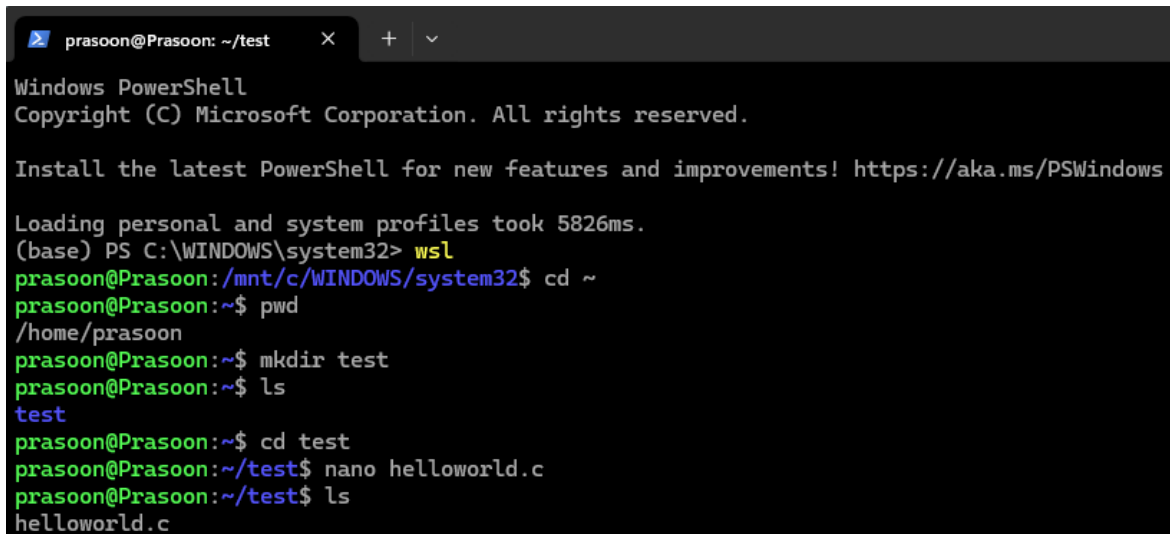
```
#include<stdio.h>

#include<stdlib.h>

int main(int argc, char *argv[]){
    int x=atoi(argv[1]);
    int y=atoi(argv[2]);
    printf("Sum = %d", x+y);
    printf("\n");
    return 0;
}
```

Screenshots

Creating a folder and .c file:



```
praseon@Praseon: ~/test
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

Loading personal and system profiles took 5826ms.
(base) PS C:\WINDOWS\system32> wsl
praseon@Praseon:/mnt/c/WINDOWS/system32$ cd ~
praseon@Praseon:~$ pwd
/home/praseon
praseon@Praseon:~$ mkdir test
praseon@Praseon:~$ ls
test
praseon@Praseon:~$ cd test
praseon@Praseon:~/test$ nano helloworld.c
praseon@Praseon:~/test$ ls
helloworld.c
```

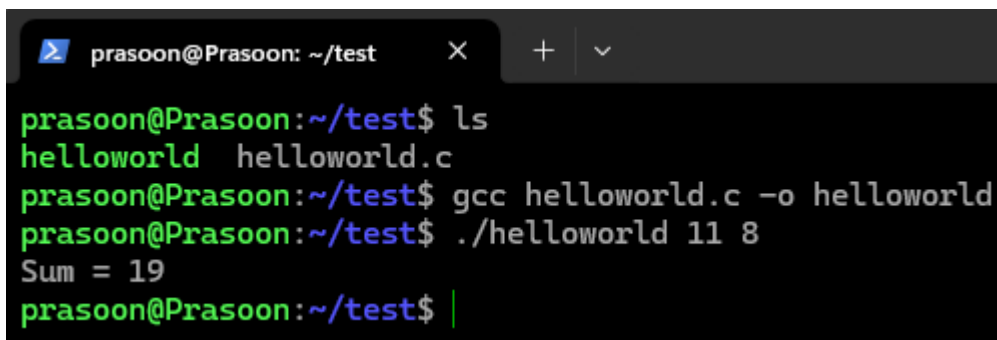
Writing program to add 2 numbers passed from CLI:



```
GNU nano 8.7.1 helloworld.c
#include<stdio.h>
#include<stdlib.h>

int main(int argc, char *argv[]){
int x=atoi(argv[1]);
int y=atoi(argv[2]);
printf("Sum = %d",x+y);
printf("\n")
return 0
}
```

Compiling and running the file:



```
praseon@Praseon:~/test$ ls
helloworld helloworld.c
praseon@Praseon:~/test$ gcc helloworld.c -o helloworld
praseon@Praseon:~/test$ ./helloworld 11 8
Sum = 19
praseon@Praseon:~/test$ |
```

Conclusion

This experiment explored the Ubuntu command-line environment through WSL and demonstrated the creation, compilation, and execution of a C program. It provided practical experience with essential Linux commands for directory navigation and file management. The program successfully accepted numbers as command-line arguments and performed their addition after converting them into integer values. The experiment improved understanding of GCC compilation, program execution, and command-line input handling in C. Hence, the objectives of the experiment were successfully achieved.